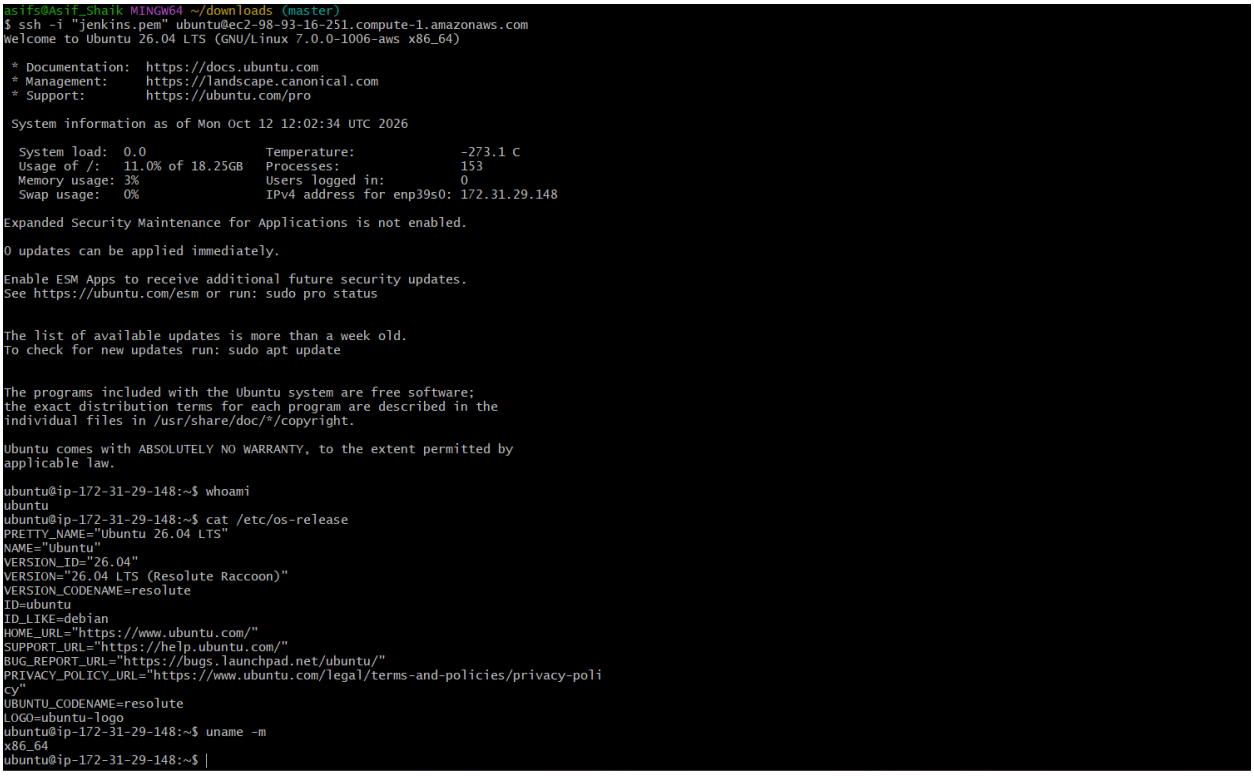
Task 1: Setup a virtual machine using Ubuntu image.

Steps Used

1. Logged in to the AWS Management Console.
2. Opened the **EC2** service and selected **Launch instances**.
3. Entered the instance name as ubuntu-docker-lab.
4. Selected **Ubuntu Server 26.04 LTS** as the Amazon Machine Image (AMI).
5. Selected the instance type m7i-flex.large with 2 vCPUs and 8 GiB RAM.
6. Selected the existing key pair jenkins for secure SSH access.
7. Selected the default VPC and enabled automatic public IP assignment.
8. Created a security group named docker-lab-sg with inbound rules for SSH (22), Nginx (81), and Jenkins (8080).
9. Configured a 20 GiB gp3 root EBS volume.
10. Reviewed the configuration and launched the EC2 instance.
11. Waited for the instance to enter the **Running** state.
12. Opened Git Bash on Windows and connected to the instance using SSH:

ssh -i "jenkins.pem" ubuntu@ec2-98-93-16-251.compute-1.amazonaws.com

13. Verified that the SSH connection was successful and the Ubuntu terminal was accessible.



Observation

- The EC2 instance was launched successfully with Ubuntu 26.04 LTS.
- The instance was accessible remotely through SSH using the jenkins.pem private key.
- The default login username was ubuntu.
- The instance received a public IPv4 address for remote connectivity.
- The Ubuntu welcome message confirmed that the operating system was running on the EC2 instance.

Conclusion

Successfully launched an Ubuntu EC2 virtual machine on AWS and established a secure SSH connection from Windows using Git Bash. The instance is ready for the next task: installing Docker and verifying its functionality.

Task 2: Install Docker on Ubuntu EC2

Steps Used

1. Updated Ubuntu package repositories using

   `sudo apt update`.

2. Installed Docker Engine using

   `sudo apt install -y docker.io`.

3. Started Docker and enabled automatic startup using

   `sudo systemctl enable --now docker`.

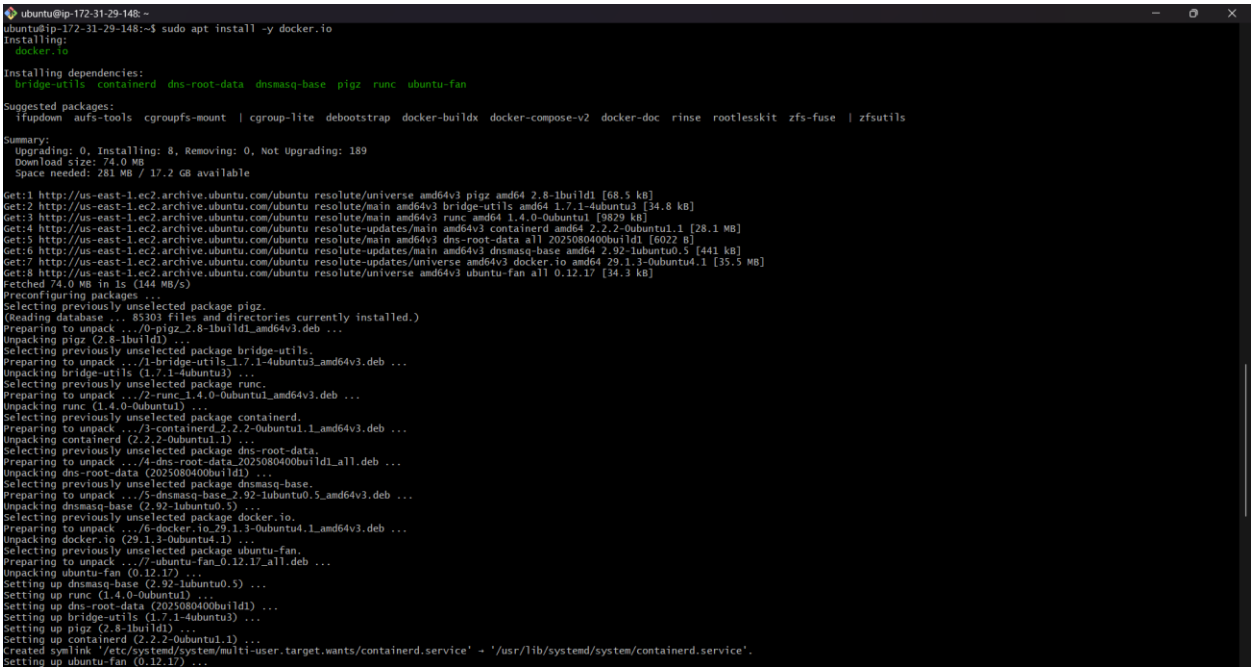
4. Verified the Docker version using

   `sudo docker --version`.

5. Checked the Docker service status using

`sudo systemctl is-active docker`.

6. Tested Docker by running the `hello-world` container using `sudo docker run --rm hello-world`.



```
ubuntu@ip-172-31-29-148:~$ sudo systemctl enable --now docker
ubuntu@ip-172-31-29-148:~$ sudo docker --version
Docker version 29.1.3, build 29.1.3-0ubuntu4.1
ubuntu@ip-172-31-29-148:~$ sudo systemctl is-active docker
active
ubuntu@ip-172-31-29-148:~$ sudo docker run --rm hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
4f55086f7dd0: Pull complete
d5e71e642bf5: Download complete
Digest: sha256:5e23090353324d887c48ad5e5c56d294eab81588df9605b07d1afe895f9cc8f8
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
    (amd64)
 3. The Docker daemon created a new container from that image which runs the
    executable that produces the output you are currently reading.
 4. The Docker daemon streamed that output to the Docker client, which sent it
    to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
 $ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
 https://hub.docker.com/

For more examples and ideas, visit:
 https://docs.docker.com/get-started/

ubuntu@ip-172-31-29-148:~$
```

Observation

- Docker was installed successfully on the Ubuntu EC2 instance.
- The Docker service was active.
- The installed Docker version was displayed.
- The `hello-world` container executed successfully and displayed the confirmation message.

Conclusion

Successfully installed and configured Docker on the Ubuntu EC2 instance. Verified that the Docker service was running and capable of downloading and executing containers. The environment is ready to pull the five required Docker images.

Task 3: Pull 5 docker images. (nginx, Apache tomcat, ubuntu, Jenkins, SonarQube)

Steps Used

1. Downloaded the Nginx image using

   `sudo docker pull nginx:latest`.

2. Downloaded the Apache Tomcat image using

   `sudo docker pull tomcat:10.1`.

3. Downloaded the Ubuntu image using

   `sudo docker pull ubuntu:24.04`.

4. Downloaded the Jenkins image using

   `sudo docker pull jenkins/jenkins:lts-jdk21`.

5. Downloaded the SonarQube image using

   `sudo docker pull sonarqube:community`.

6. Verified the downloaded images using

   `sudo docker images`.

```
ubuntu@ip-172-31-29-148: ~
ubuntu@ip-172-31-29-148:~$ sudo docker pull nginx:latest
latest: Pulling from library/nginx
807ab930f3a0: Pull complete
ecc510c1e359: Pull complete
947d22046304: Pull complete
04bdbc7ea6b5: Pull complete
f7e1db57b58b: Pull complete
8020170dc65f: Pull complete
1efc53192927: Pull complete
a1b85ab28c90: Download complete
c329293b1d90: Download complete
Digest: sha256:f9ea18bfa4fad859e1ed38259d711da7ccad2c3516e875cec3351a57c859f571
Status: Downloaded newer image for nginx:latest
docker.io/library/nginx:latest
ubuntu@ip-172-31-29-148:~$
ubuntu@ip-172-31-29-148:~$ sudo docker pull tomcat:10.1
10.1: Pulling from library/tomcat
4f4fb700ef54: Pull complete
4e07a0f12b2c: Pull complete
06ad70e463aa: Pull complete
5dec61325458: Pull complete
f39241fdc590: Pull complete
414facebfbee: Pull complete
130739ca07b1: Pull complete
0b8e7b5ef7b3: Pull complete
f678d102d0fc: Download complete
39551458a857: Download complete
Digest: sha256:a32181602d8667be0185bdb3f17d2b7491777dd2d51f8969443a90a277068232
Status: Downloaded newer image for tomcat:10.1
docker.io/library/tomcat:10.1
ubuntu@ip-172-31-29-148:~$ sudo docker pull ubuntu:24.04
24.04: Pulling from library/ubuntu
a60ef8ec184e: Pull complete
2ea90c0e4faa: Download complete
Digest: sha256:534baea6a22c03a63003dbc8dbe78fe34bc0d7e595d9a9dc9834884ff530eb55
Status: Downloaded newer image for ubuntu:24.04
docker.io/library/ubuntu:24.04
ubuntu@ip-172-31-29-148:~$ sudo docker pull jenkins/jenkins:lts-jdk21
lts-jdk21: Pulling from jenkins/jenkins
a91427fe0e3f: Pull complete
6eefb2f5d3e9: Pull complete
408777b27aa9: Pull complete
810008851f3a: Pull complete
d89089c8a5a3: Pull complete
701ded2f9b33: Pull complete
968c36a3b2c5: Pull complete
3dd4a99993ab: Pull complete
a20a14499a3e: Pull complete
463a574793f8: Pull complete
f9e31fab50e8: Pull complete
95bc7464aaea: Pull complete
44136fa355b3: Download complete
9613b4f6b0c7: Download complete
Digest: sha256:a660310e39ade10631f774bacd5219de767dfd08947ea5c32a739b5e5bb382c1
Status: Downloaded newer image for jenkins/jenkins:lts-jdk21
```

```
ubuntu@ip-172-31-29-148:~$ sudo docker pull sonarqube:community
community: Pulling from library/sonarqube
4f4fb700ef54: Pull complete
065adb12f4f5: Pull complete
96fe3bd0148e: Pull complete
03217b7e6be2: Pull complete
54929e68a7af: Pull complete
0a66c41acdcb: Pull complete
780212bdfe24: Download complete
6cfeebcd7dea: Download complete
Digest: sha256:c0f1160bccfa435db4168c2d7df69af3c3ea7bfeb87395613014048fb33a68c1
Status: Downloaded newer image for sonarqube:community
docker.io/library/sonarqube:community
```

```
ubuntu@ip-172-31-29-148:~$ sudo docker images

IMAGE                      ID             DISK USAGE   CONTENT SIZE   EXTRA
hello-world:latest         5e2309035332       25.9kB         9.49kB
jenkins/jenkins:lts-jdk21  a660310e39ad        720MB          246MB
nginx:latest               f9ea18bfa4fa        243MB         66.7MB
sonarqube:community        c0f1160bccfa       2.55GB          1.1GB
tomcat:10.1                a32181602d86        640MB          173MB
ubuntu:24.04               534baea6a22c        119MB         31.7MB
ubuntu@ip-172-31-29-148:~$
```

Repository	Tag

nginx	latest
tomcat	10.1
ubuntu	24.04
jenkins/jenkins	lts-jdk21
sonarqube	community

Observation

- Docker downloaded the five requested images from their registries.
- The images appeared in the local Docker image list with their respective repository names and tags.
- No containers were started during this task.

Conclusion

Successfully downloaded and verified the five Docker images: Nginx, Apache Tomcat, Ubuntu, Jenkins, and SonarQube. The images are ready to be used to create containers in the following tasks.

Task 4: Run Nginx Container and Expose on Port 81

Steps Used

1. Created and started the Nginx container using

   `sudo docker run -d --name nginx-lab -p 81:80 nginx:latest`.

2. Verified that the container was running using

   `sudo docker ps`.

3. Tested the Nginx web server from the EC2 terminal using

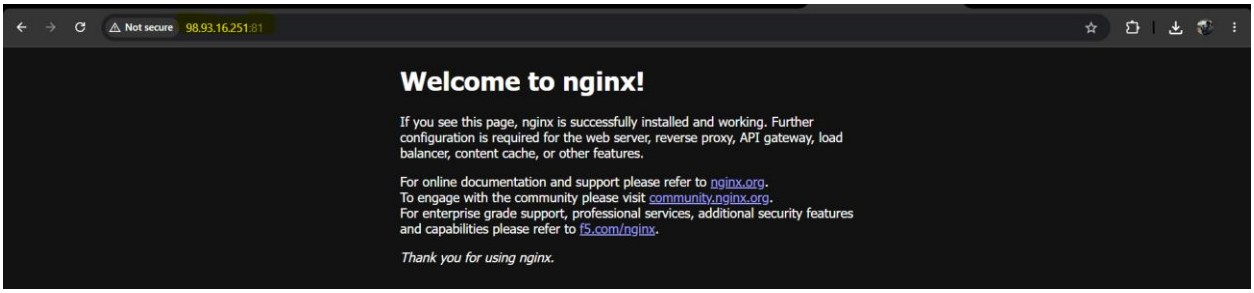
   `curl` http://localhost:81.

4. Accessed the Nginx web server through the EC2 public IPv4 address on port 81.

```
ubuntu@ip-172-31-29-148:~$ sudo docker run -d --name nginx-lab -p 81:80 nginx:latest
91a5dfdcb3fb0e0dfd8cb307c4df4b42a913e0c70ad07360d4d2d4fdb4c20968
ubuntu@ip-172-31-29-148:~$
ubuntu@ip-172-31-29-148:~$ sudo docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED         STATUS         PORTS                                   NAMES
91a5dfdcb3fb   nginx:latest   "/docker-entrypoint.…"   23 seconds ago   Up 22 seconds   0.0.0.0:81->80/tcp, [::]:81->80/tcp   nginx-lab
ubuntu@ip-172-31-29-148:~$ curl http://localhost:81
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, nginx is successfully installed and working.
Further configuration is required for the web server, reverse proxy,
API gateway, load balancer, content cache, or other features.</p>

<p>For online documentation and support please refer to
<a href="https://nginx.org/">nginx.org</a>.<br/>
To engage with the community please visit
<a href="https://community.nginx.org/">community.nginx.org</a>.<br/>
For enterprise grade support, professional services, additional
security features and capabilities please refer to
<a href="https://f5.com/nginx">f5.com/nginx</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
ubuntu@ip-172-31-29-148:~$
```



Observation

- The Nginx container started successfully.
- Docker mapped host port 81 to container port 80.
- The Nginx welcome page was returned when the service was tested.

Conclusion

Successfully deployed Nginx in a Docker container and exposed the web server on port 81 of the EC2 instance. Verified that the web server was accessible through the configured port mapping.

Task 5: Delete the Apache Tomcat Image Locally

Steps Used

1. Listed local Docker images using

   `sudo docker images`.

2. Checked existing containers using

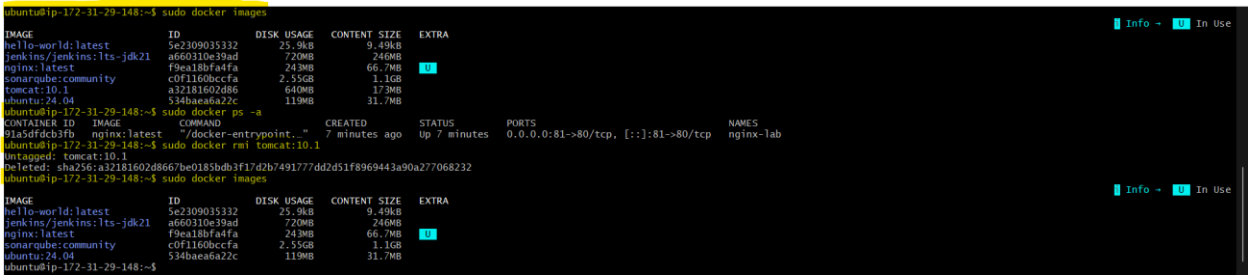
   `sudo docker ps -a`.

3. Removed the Apache Tomcat image using

   `sudo docker rmi tomcat:10.1`.

4. Verified the deletion by running

   `sudo docker images` again.



Observation

- The Apache Tomcat image was identified in the local Docker image list.
- The image was removed successfully, provided no remaining container depended on it.
- The other required images remained available locally.

Conclusion

Successfully removed the Apache Tomcat image from local Docker storage. This task demonstrated Docker image management using the `docker rmi` command.

Task 6: Inspect Jenkins and SonarQube Docker Images

Steps Used

1. Inspected the Jenkins image using

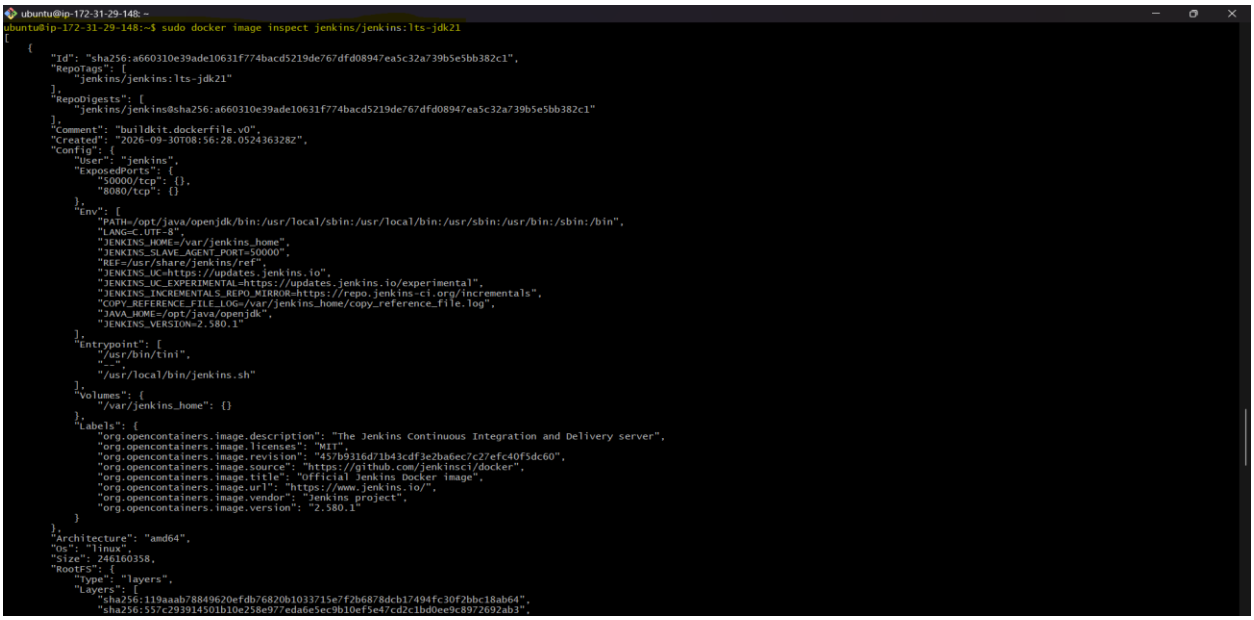
   `sudo docker image inspect jenkins/jenkins:lts-jdk21`.

2. Displayed the Jenkins image ID, exposed ports, and working directory using Docker's format option.
3. Inspected the SonarQube image using

   `sudo docker image inspect sonarqube:community`.

4. Displayed the SonarQube image ID, exposed ports, and working directory.
5. Verified the images using

   `sudo docker images`.



```
ubuntu@ip-172-31-29-148:~$ sudo docker image inspect \
  --format 'Image ID: {{.Id}}' \
  jenkins/jenkins:lts-jdk21
Image ID: sha256:a660310e39ade10631f774bacd5219de767dfd08947ea5c32a739b5e5bb382c1
ubuntu@ip-172-31-29-148:~$
ubuntu@ip-172-31-29-148:~$ sudo docker image inspect \
  --format 'Exposed Ports: {{json .Config.ExposedPorts}}' \
  jenkins/jenkins:lts-jdk21
Exposed Ports: {"50000/tcp":{},"8080/tcp":{}}
ubuntu@ip-172-31-29-148:~$
ubuntu@ip-172-31-29-148:~$ sudo docker image inspect \
  --format 'Working Directory: {{.Config.WorkingDir}}' \
  jenkins/jenkins:lts-jdk21
Working Directory:
ubuntu@ip-172-31-29-148:~$
```

```
ubuntu@ip-172-31-29-148: ~
ubuntu@ip-172-31-29-148:~$ sudo docker image inspect sonarqube:community
[
    {
        "Id": "sha256:c0f1160bccfa435db4168c2d7df69af3c3ea7bfeb87395613014048fb33a68c1",
        "RepoTags": [
            "sonarqube:community"
        ],
        "RepoDigests": [
            "sonarqube@sha256:c0f1160bccfa435db4168c2d7df69af3c3ea7bfeb87395613014048fb33a68c1"
        ],
        "Comment": "buildkit.dockerfile.v0",
        "Created": "2026-10-02T02:34:24.202145562Z",
        "Config": {
            "User": "sonarqube",
            "ExposedPorts": {
                "9000/tcp": {}
            },
            "Env": [
                "PATH=/opt/java/openjdk/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin",
                "JAVA_HOME=/opt/java/openjdk",
                "LANG=en_US.UTF-8",
                "LANGUAGE=en_US:en",
                "LC_ALL=en_US.UTF-8",
                "JAVA_VERSION=jdk-25.0.4.1+1",
                "DOCKER_RUNNING=true",
                "SONARQUBE_HOME=/opt/sonarqube",
                "SONAR_VERSION=26.9.0.129388",
                "SQ_DATA_DIR=/opt/sonarqube/data",
                "SQ_EXTENSIONS_DIR=/opt/sonarqube/extensions",
                "SQ_LOGS_DIR=/opt/sonarqube/logs",
                "SQ_TEMP_DIR=/opt/sonarqube/temp",
                "ES_TMPDIR=/opt/sonarqube/temp"
            ],
            "Entrypoint": [
                "/opt/sonarqube/docker/entrypoint.sh"
            ],
            "Volumes": {
                "/opt/sonarqube/data": {},
                "/opt/sonarqube/extensions": {},
                "/opt/sonarqube/logs": {},
                "/opt/sonarqube/temp": {}
            },
            "WorkingDir": "/opt/sonarqube",
            "Labels": {
                "io.k8s.description": "SonarQube Community Build is a self-managed, automatic code review tool that systematically helps you deliver Clean Code.",
                "io.openshift.min-cpu": "400m",
                "io.openshift.min-memory": "2048M",
                "io.openshift.non-scalable": "true",
                "io.openshift.tags": "sonarqube,static-code-analysis,code-quality,clean-code",
                "org.opencontainers.image.url": "https://github.com/SonarSource/docker-sonarqube",
                "org.opencontainers.image.version": "24.04"
            },
            "StopSignal": "SIGINT"
        },
        "Architecture": "amd64",
        "Os": "linux",
        "Size": 1090884200,
        "RootFS": {
```

```
ubuntu@ip-172-31-29-148:~$ sudo docker image inspect \
  --format 'Image ID: {{.Id}}' \
  sonarqube:community
Image ID: sha256:c0f1160bccfa435db4168c2d7df69af3c3ea7bfeb87395613014048fb33a68c1
ubuntu@ip-172-31-29-148:~$
ubuntu@ip-172-31-29-148:~$ sudo docker image inspect \
  --format 'Exposed Ports: {{json .Config.ExposedPorts}}' \
  sonarqube:community
Exposed Ports: {"9000/tcp":{}}
ubuntu@ip-172-31-29-148:~$
ubuntu@ip-172-31-29-148:~$ sudo docker image inspect \
  --format 'Working Directory: {{.Config.WorkingDir}}' \
  sonarqube:community
Working Directory: /opt/sonarqube
ubuntu@ip-172-31-29-148:~$
ubuntu@ip-172-31-29-148:~$ sudo docker images
                                                                                                    Info →  U  In Use
IMAGE                          ID             DISK USAGE   CONTENT SIZE   EXTRA
hello-world:latest             5e2309035332       25.9kB          9.49kB
jenkins/jenkins:lts-jdk21      a660310e39ad        720MB           246MB
nginx:latest                   f9ea18bfa4fa        243MB          66.7MB    U
sonarqube:community            c0f1160bccfa       2.55GB           1.1GB
ubuntu:24.04                   534baea6a22c        119MB          31.7MB
ubuntu@ip-172-31-29-148:~$
```

Observation

- Docker displayed detailed metadata for both images.
- The image IDs and configuration information were available.
- Jenkins commonly uses container port 8080 for its web interface.
- SonarQube commonly uses container port 9000 for its web interface.
- Both images remained in local Docker storage.

Conclusion

Successfully inspected the Jenkins and SonarQube Docker images and examined their configuration metadata. This task demonstrated how to use Docker's image inspection commands before creating containers.

Task 7: Run Jenkins Container and Execute a Sample Job

Steps Used

1. Created a persistent Docker volume using

   `sudo docker volume create jenkins_home`.

2. Started the Jenkins container using the

   `jenkins/jenkins:lts-jdk21` image and mapped port 8080.

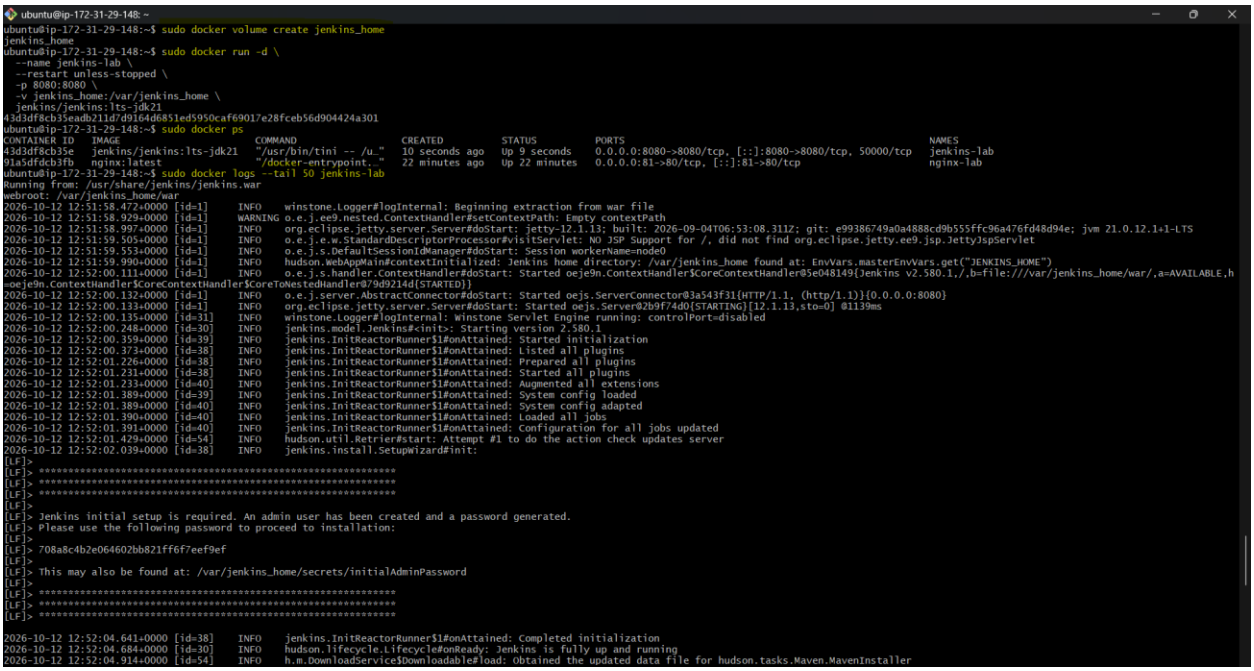
3. Verified the running container using

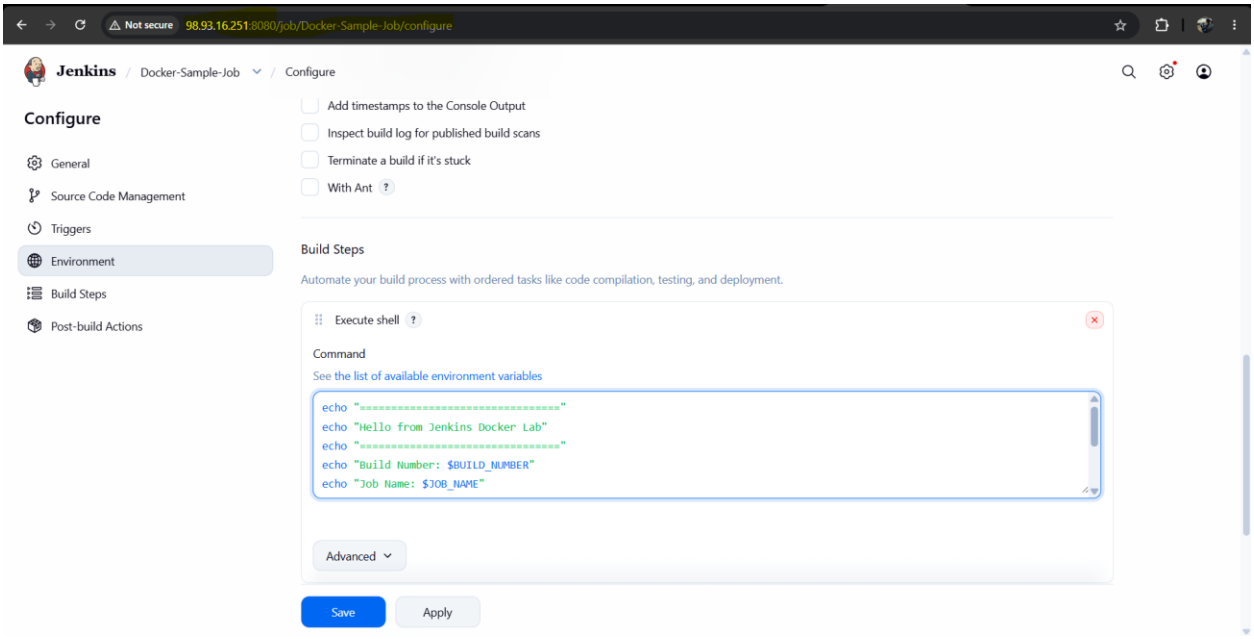
   `sudo docker ps`.

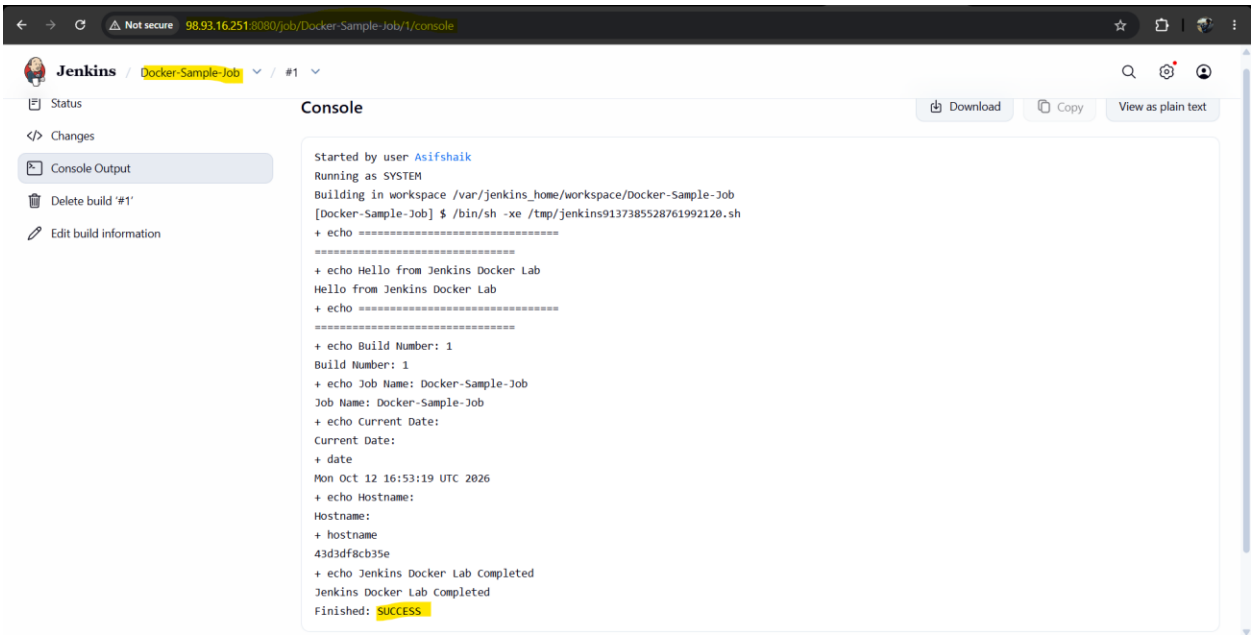
4. Checked the container startup logs using

   `sudo docker logs --tail 50 jenkins-lab`.

5. Retrieved the initial administrator password from the Jenkins container.
6. Accessed the Jenkins web interface through the EC2 public IP on port 8080.
7. Completed the initial Jenkins setup and created a Freestyle project named `Docker-Sample-Job`.
8. Configured an Execute shell build step containing basic Linux commands and Jenkins environment variables.
9. Executed the job using Build Now and reviewed its Console Output.


ubuntu@ip-172-31-29-148: ~
ubuntu@ip-172-31-29-148:~$ sudo docker volume create jenkins_home
jenkins_home
ubuntu@ip-172-31-29-148:~$ sudo docker run -d \
--name jenkins-lab \
--restart unless-stopped \
-p 8080:8080 \
-v jenkins_home:/var/jenkins_home \
jenkins/jenkins:lts-jdk21
43d3df8cb35eadb211d7d9164d6851ed5950caf69017e28fceb56d904424a301
ubuntu@ip-172-31-29-148:~$ sudo docker ps
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES
43d3df8cb35e jenkins/jenkins:lts-jdk21 "/usr/bin/tini -- /u…" 10 seconds ago Up 9 seconds 0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp, 50000/tcp jenkins-lab
01a5dfdcb3fb nginx:latest "/docker-entrypoint.…" 22 minutes ago Up 22 minutes 0.0.0.0:81->80/tcp, [::]:81->80/tcp nginx-lab
ubuntu@ip-172-31-29-148:~$ sudo docker logs --tail 50 jenkins-lab
Running from: /usr/share/jenkins/jenkins.war
webroot: /var/jenkins_home/war
2026-10-12 12:51:58.472+0000 [id=1] INFO winstone.Logger#logInternal: Beginning extraction from war file
2026-10-12 12:51:58.929+0000 [id=1] WARNING o.e.j.e9.nested.ContextHandler#setContextPath: Empty contextPath
2026-10-12 12:51:58.997+0000 [id=1] INFO org.eclipse.jetty.server.Server#doStart: jetty-12.1.13; built: 2026-09-04T06:53:08.311Z; git: e99386749a0a4888cd9b555ffc96a476fd48d94e; jvm 21.0.12.1+1-LTS
2026-10-12 12:51:59.505+0000 [id=1] INFO o.e.j.e.w.StandardDescriptorProcessor#visitServlet: NO JSP Support for /, did not find org.eclipse.jetty.ee9.jsp.JettyJspServlet
2026-10-12 12:51:59.553+0000 [id=1] INFO o.e.j.s.DefaultSessionIdManager#doStart: Session workerName=node0
2026-10-12 12:51:59.990+0000 [id=1] INFO hudson.WebAppMain#contextInitialized: Jenkins home directory: /var/jenkins_home found at: EnvVars.masterEnvVars.get("JENKINS_HOME")
2026-10-12 12:52:00.111+0000 [id=1] INFO o.e.j.s.handler.ContextHandler#doStart: Started oeje9n.ContextHandler$CoreContextHandler@5e048149{Jenkins v2.580.1,/,b=file:///var/jenkins_home/war/,a=AVAILABLE,h
=oeje9n.ContextHandler$CoreContextHandler$CoreToNestedHandler@79d9214d{STARTED}}
2026-10-12 12:52:00.132+0000 [id=1] INFO o.e.j.server.AbstractConnector#doStart: Started oejs.ServerConnector@3a543f31{HTTP/1.1, (http/1.1)}{0.0.0.0:8080}
2026-10-12 12:52:00.133+0000 [id=1] INFO org.eclipse.jetty.server.Server#doStart: Started oejs.Server@2b9f74d0{STARTING}[12.1.13,sto=0] @1139ms
2026-10-12 12:52:00.135+0000 [id=31] INFO winstone.Logger#logInternal: Winstone Servlet Engine running: controlPort=disabled
2026-10-12 12:52:00.248+0000 [id=30] INFO jenkins.model.Jenkins#<init>: Starting version 2.580.1
2026-10-12 12:52:00.359+0000 [id=39] INFO jenkins.InitReactorRunner$1#onAttained: Started initialization
2026-10-12 12:52:00.373+0000 [id=38] INFO jenkins.InitReactorRunner$1#onAttained: Listed all plugins
2026-10-12 12:52:01.226+0000 [id=38] INFO jenkins.InitReactorRunner$1#onAttained: Prepared all plugins
2026-10-12 12:52:01.231+0000 [id=38] INFO jenkins.InitReactorRunner$1#onAttained: Started all plugins
2026-10-12 12:52:01.233+0000 [id=40] INFO jenkins.InitReactorRunner$1#onAttained: Augmented all extensions
2026-10-12 12:52:01.389+0000 [id=39] INFO jenkins.InitReactorRunner$1#onAttained: System config loaded
2026-10-12 12:52:01.389+0000 [id=40] INFO jenkins.InitReactorRunner$1#onAttained: System config adapted
2026-10-12 12:52:01.390+0000 [id=40] INFO jenkins.InitReactorRunner$1#onAttained: Loaded all jobs
2026-10-12 12:52:01.391+0000 [id=40] INFO jenkins.InitReactorRunner$1#onAttained: Configuration for all jobs updated
2026-10-12 12:52:01.429+0000 [id=54] INFO hudson.util.Retrier#start: Attempt #1 to do the action check updates server
2026-10-12 12:52:02.039+0000 [id=38] INFO jenkins.install.SetupWizard#init:
[LF]>
[LF]> **
[LF]> **
[LF]> **
[LF]>
[LF]> Jenkins initial setup is required. An admin user has been created and a password generated.
[LF]> Please use the following password to proceed to installation:
[LF]>
[LF]> 708a8c4b2e064602bb821ff6f7eef9ef
[LF]>
[LF]> This may also be found at: /var/jenkins_home/secrets/initialAdminPassword
[LF]>
[LF]> **
[LF]> **
[LF]> **
2026-10-12 12:52:04.641+0000 [id=38] INFO jenkins.InitReactorRunner$1#onAttained: Completed initialization
2026-10-12 12:52:04.684+0000 [id=30] INFO hudson.lifecycle.Lifecycle#onReady: Jenkins is fully up and running
2026-10-12 12:52:04.914+0000 [id=54] INFO h.m.DownloadService$Downloadable#load: Obtained the updated data file for hudson.tasks.Maven.MavenInstaller


Not secure 98.93.16.251:8080/job/Docker-Sample-Job/configure
Jenkins / Docker-Sample-Job / Configure
Configure
General
Source Code Management
Triggers
Environment
Build Steps
Post-build Actions
Add timestamps to the Console Output
Inspect build log for published build scans
Terminate a build if it's stuck
With Ant
Build Steps
Automate your build process with ordered tasks like code compilation, testing, and deployment.
Execute shell
Command
See the list of available environment variables
echo "================================"
echo "Hello from Jenkins Docker Lab"
echo "================================"
echo "Build Number: $BUILD_NUMBER"
echo "Job Name: $JOB_NAME"
Advanced
Save
Apply



Observation

- Jenkins ran successfully inside a Docker container.
- The Jenkins web interface was accessible through port 8080.
- A Freestyle project was created and executed.
- The console output displayed the build number, job name, date, and hostname.
- The build completed with `Finished: SUCCESS`.

Conclusion

Successfully deployed Jenkins as a Docker container and executed a sample Freestyle project. This task demonstrated container deployment, port mapping, persistent storage, Jenkins configuration, and basic build execution.